Assignment 17.1

R NIKHIL KUMAR

2303A52260

Batch: 44

Task 1: Customer Data Cleaning

Prompt:

Create a pandas script to clean customer data: fill missing values (age, income, city), remove duplicates, lowercase city names, and encode gender and city.

Code:

```python
def task1_customer_data_cleaning(file_path: str) -> pd.DataFrame:
    """
    Task 1:
    - Handle missing values in age, income, city
    - Remove duplicates
    - Standardize city names to lowercase
    - Encode categorical variables (gender, city)
    """
    df = pd.read_csv(file_path)

    age_col = _find_column(df, ["age"])
    income_col = _find_column(df, ["income", "purchase amount", "purchase amount usd", "salary"])
    city_col = _find_column(df, ["city", "location", "town"])
    gender_col = _find_column(df, ["gender", "sex"])

    if age_col:
        _fill_numeric_with_median(df, age_col)
    if income_col:
        _fill_numeric_with_median(df, income_col)
    if city_col:
        _fill_categorical_with_mode(df, city_col)
        df[city_col] = df[city_col].astype(str).str.strip().str.lower()

    df = df.drop_duplicates().copy()

    encode_cols = [col for col in [gender_col, city_col] if col is not None]
    if encode_cols:
        df = pd.get_dummies(df, columns=encode_cols, drop_first=False)

    return df
```

Output:

```
age,income,gender_Female,gender_Male,city_chicago,city_los angeles,city_new york
25.0,50000.0,False,True,False,False,True
30.0,60000.0,True,False,False,True,False
30.0,60000.0,False,True,True,False,False
45.0,80000.0,True,False,True,False,False
```

Observation:

Missing values were filled (age and income became numeric values, city gaps were filled), duplicate rows were removed, and city/gender were encoded into separate indicator columns.
Observation: the output is analysis-ready with no raw text categories left for gender/city.

Task 2: Sales Data Preprocessing

Prompt:

Create a pandas script to preprocess sales data: convert dates, extract month/year/day, normalize sales, and remove outliers.

Code:

```python
def task2_sales_data_preprocessing(file_path: str) -> pd.DataFrame:
    """
    Task 2:
    - Convert date column to datetime
    - Create month, year, day_of_week features
    - Normalize sales amount
    - Detect and handle outliers
    """
    df = pd.read_csv(file_path)

    date_col = _find_column(df, ["date", "order date", "transaction date"])
    sales_col = _find_column(df, ["sales amount", "sales", "amount", "revenue", "total sales"])

    if date_col is None:
        raise ValueError("No date column found. Try one of: date, order date, transaction date.")

    df[date_col] = pd.to_datetime(df[date_col], errors="coerce")
    df = df.dropna(subset=[date_col]).copy()

    df["month"] = df[date_col].dt.month
    df["year"] = df[date_col].dt.year
    df["day_of_week"] = df[date_col].dt.day_name()

    if sales_col is None:
        raise ValueError("No sales amount column found. Try one of: sales amount, sales, amount, revenue.")

    df[sales_col] = pd.to_numeric(df[sales_col], errors="coerce")
    df = df.dropna(subset=[sales_col]).copy()

    df = _remove_outliers_iqr(df, sales_col)

    scaler = MinMaxScaler()
    df[f"{sales_col}_normalized"] = scaler.fit_transform(df[[sales_col]])

    return df
```

Output:

```csv
date,sales amount,region,month,year,day_of_week,sales amount_normalized
2025-01-01,1000,North,1,2025,Wednesday,0.0
2025-01-02,1200,South,1,2025,Thursday,1.0
2025-01-04,1100,West,1,2025,Saturday,0.5
```

Observation:

Date values were converted and expanded into month, year, and day of week, invalid dates were dropped, extreme sales outliers were removed, and sales were min-max normalized.
Observation: normalization worked as expected (values scaled between 0 and 1), and the high outlier row was excluded.

Task 3: Healthcare Data Preparation

Prompt:

Create a pandas and sklearn script to preprocess healthcare data: fill missing values, encode categories, scale features, and split into train/test sets.

Code:

```python
def task3_healthcare_data_preparation(
    file_path: str,
    scale_method: str = "standard",
    test_size: float = 0.2,
    random_state: int = 42,
) -> Tuple[pd.DataFrame, pd.DataFrame, pd.Series, pd.Series, pd.DataFrame]:
    """
    Task 3:
    - Handle missing values in blood_pressure, cholesterol
    - Encode categorical features
    - Scale numeric features
    - Split into training and testing sets
    """
    df = pd.read_csv(file_path)

    bp_col = _find_column(df, ["blood_pressure", "blood pressure", "bp"])
    chol_col = _find_column(df, ["cholesterol", "chol"])

    for col in [bp_col, chol_col]:
        if col:
            _fill_numeric_with_median(df, col)

    categorical_cols = df.select_dtypes(include=["object", "category", "bool"]).columns.tolist()
    if categorical_cols:
        df = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

    numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()

    target_col = _find_column(df, ["risk", "target", "outcome", "label", "class"])
    if target_col is None and numeric_cols:
        target_col = numeric_cols[-1]

    if target_col is None:
        raise ValueError("Could not identify target column.")

    X = df.drop(columns=[target_col])
    y = df[target_col]

    num_features = X.select_dtypes(include=[np.number]).columns.tolist()
    if num_features:
        scaler = StandardScaler() if scale_method == "standard" else MinMaxScaler()
        X.loc[:, num_features] = scaler.fit_transform(X[num_features])

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=random_state
    )

    processed_df = pd.concat([X, y], axis=1)
    return X_train, X_test, y_train, y_test, processed_df
```

Output:

```csv
blood_pressure,cholesterol,smoker_yes,exercise_low,exercise_medium,risk
-1.414213562373095,-1.0,True,True,False,1
1.414213562373095,-0.3333333333333333,False,False,False,0
0.0,1.6666666666666667,True,False,True,1
0.0,-0.3333333333333333,False,True,False,0
```

Observation:

Missing blood pressure and cholesterol values were imputed, categorical variables were encoded, and numeric features were scaled (standard scaling).
Observation: scaled features are centred and comparable in magnitude, making the dataset suitable for ML training/testing workflows.

Task 4: Employee Salary Data Preprocessing

Prompt:

Create a pandas and sklearn script to clean salary data: handle missing values, remove outliers, lowercase department names, and encode department and location.

Code:

```python
def task4_employee_salary_preprocessing(file_path: str) -> pd.DataFrame:
    """
    Task 4:
    - Handle missing values
    - Detect and remove salary outliers
    - Standardize department names (lowercase)
    - Label encode job location and department
    """
    df = pd.read_csv(file_path)

    salary_col = _find_column(df, ["salary", "monthly salary", "annual salary", "pay"])
    department_col = _find_column(df, ["department", "dept"])
    location_col = _find_column(df, ["job location", "location", "city", "work location"])

    for col in df.columns:
        if pd.api.types.is_numeric_dtype(df[col]):
            _fill_numeric_with_median(df, col)
        else:
            _fill_categorical_with_mode(df, col)

    if salary_col:
        df[salary_col] = pd.to_numeric(df[salary_col], errors="coerce")
        df = df.dropna(subset=[salary_col]).copy()
        df = _remove_outliers_iqr(df, salary_col)

    if department_col:
        df[department_col] = df[department_col].astype(str).str.strip().str.lower()

    for col in [location_col, department_col]:
        if col:
            encoder = LabelEncoder()
            df[col] = encoder.fit_transform(df[col].astype(str))

    return df
```

Output:

```
salary,department,job location,experience
50000.0,1,3,3
52000.0,3,2,4
51000.0,2,0,5
51500.0,0,1,2
```

Observation:

Missing values were handled, salary outliers were removed, department text was standardized to lowercase before encoding, and department/location were label-encoded.
Observation: the extreme salary value was removed, and categorical HR fields were converted to numeric format for analytics models.